Enterprise MMM Decision Engine

Complete Technical & Conceptual Documentation

Covers: Bayesian Statistics · PyMC · PyMC-Marketing · Application Architecture

Version 1.0 · April 2026

Table of Contents

Table of Contents 2
1. What Is Marketing Mix Modelling (MMM)? 5
1.1 Core Concepts 5
Baseline vs. Incremental Sales 5
Adstock (Carryover Effect) 5
Saturation (Diminishing Returns) 5
Response Curves 6
1.2 Traditional MMM vs. Bayesian MMM 6
2. Bayesian Statistics — The Mathematical Engine 7
2.1 Bayes' Theorem 7
2.2 Prior Distributions 7
2.3 Likelihood Function 7
2.4 Posterior Inference via MCMC 8
No-U-Turn Sampler (NUTS) 8
Convergence Diagnostics 8
2.5 Posterior Predictive Checks 8
2.6 Model Comparison Metrics 9
3. PyMC and PyMC-Marketing 10
3.1 PyMC — Probabilistic Programming 10
Example: Defining a PyMC model 10
3.2 ArviZ — Posterior Analysis 10
3.3 PyMC-Marketing — MMM Layer 10
The MMM Class 11
Key MMM Methods 11
BudgetOptimizer 11
4. System Design & Architecture 12
4.1 Design Principles 12
4.2 Project Structure 12
4.3 Data Flow 13
5. Module-by-Module Reference 14
5.1 config/settings.py — Configuration Management 14
5.2 src/data/ — Data Pipeline 14
loader.py — load_file(source) 14
validator.py — validate_raw(df) / validate_model_df(df, channel_cols) 14
preprocessor.py — prepare_data(df) → (model_df, channel_cols) 15
5.3 src/model/ — Model Training & Evaluation 15

trainer.py — train_mmm(df, channel_cols) → (mmm, idata) 15
evaluator.py — full_evaluation_report(mmm, idata, df, target_col) 15
5.4 src/inference/ — Posterior Analysis 16
contributions.py 16
diagnostics.py 16
roas.py 16
5.5 src/optimization/ — Budget Optimisation 16
constraints.py — build_constraints(channel_cols, overrides) 16
optimizer.py — run_optimization(mmm, total_budget, bounds) → pd.DataFrame 16
5.6 src/scenario/ — Scenario Planning 17
planner.py — generate_scenario(df, multipliers, periods) → pd.DataFrame 17
simulate_scenario(mmm, scenario_df) → array 17
build_scenario_summary(scenario_df, preds, target_col, channel_cols) → pd.DataFrame 17
5.7 src/risk/ — Uncertainty Quantification 17
uncertainty.py 17
5.8 src/export/ — Results Export 18
exporter.py 18
6. UI Components — Streamlit Interface 19
6.1 Session State Schema 19
6.2 Component Summary 19
7. Utilities 21
7.1 utils/logger.py — Centralised Logging 21
7.2 utils/helpers.py 21
8. Test Suite 22
8.1 Test Coverage Summary 22
8.2 Running Tests 22
8.3 Test Design Patterns 22
9. Configuration & Deployment 24
9.1 Environment Variables (.env.example) 24
9.2 Installation 24
9.3 Makefile Commands 24
10. Step-by-Step Usage Guide 26
10.1 Preparing Your Data 26
10.2 Training the Model 26
10.3 Interpreting Insights 26
10.4 Running Scenarios 27
10.5 Optimising the Budget 27
11. Glossary 28
12. Quick Reference Card 31

Public API — One-Line Summaries 31
Common Troubleshooting 31

PART 1 — Conceptual Foundations

1. What Is Marketing Mix Modelling (MMM)?

Marketing Mix Modelling (MMM) is a statistical analysis technique used to estimate the causal impact of marketing activities on business outcomes such as sales, revenue, or clicks. It originated in the 1960s with consumer-packaged-goods companies who needed to understand the return on each pound spent across advertising channels.

MMM answers three critical business questions:

- Attribution: Which channels are driving conversions, and by how much?
- Efficiency: What is the marginal return on additional spend in each channel?
- Planning: How should I allocate my budget to maximise future returns?

1.1 Core Concepts

Baseline vs. Incremental Sales

Total observed sales are decomposed into two parts:

- Baseline: Sales that would occur without any marketing activity — driven by organic demand, seasonality, pricing, and macroeconomic factors.
- Incremental (Lift): The additional sales attributable directly to media investment.

> MMM Goal: Baseline + Channel_1_Contribution + Channel_2_Contribution + ... + ε = Observed Target

Adstock (Carryover Effect)

Advertising creates a memory effect — exposure to an ad today influences purchasing behaviour days or weeks later. Adstock models this decay mathematically. The geometric decay model is:

```
adstock_t = spend_t + α × adstock_(t-1)

Where:
  α  ∈ [0, 1] — the decay/retention rate (higher α = longer carryover)
  t  — current time period
```

In PyMC-Marketing, the adstock is implemented with a configurable maximum lag (adstock_max_lag), controlling how many past periods are included in the weighted sum.

Saturation (Diminishing Returns)

Media channels exhibit diminishing returns: doubling spend does not double sales. The Hill transformation (logistic saturation) models this S-curve or concave relationship:

```
saturation(x) = x^α / (x^α + K^α)

Where:
  x  — adstock-transformed spend
  α  — shape parameter (steepness of curve)
  K  — half-saturation point (spend level at 50% max effect)
```

Response Curves

Response curves plot the estimated marginal return (incremental outcome per £1 additional spend) as a function of total spend. They are essential for identifying the point of diminishing returns and informing budget reallocation decisions.

1.2 Traditional MMM vs. Bayesian MMM

Dimension	Comparison
Frequentist MMM	Bayesian MMM (This Engine)
Point estimates only — single 'best guess' value	Full posterior distributions — probability over all plausible values
Confidence intervals are often misinterpreted	Credible intervals have natural, probabilistic interpretation
Cannot incorporate prior knowledge	Prior beliefs encoded directly; updated by data
No uncertainty propagation to decisions	Uncertainty flows through to budget recommendations
Sensitive to multicollinearity between channels	Priors act as regularisation, stabilising estimates
Requires large sample sizes	Works with small datasets via informative priors

2. Bayesian Statistics — The Mathematical Engine

2.1 Bayes' Theorem

Bayes' Theorem describes how to update beliefs in the light of new evidence. It is the mathematical foundation of everything in this system:

```
P(θ | Data) ∝ P(Data | θ) × P(θ)

Where:
  θ        — model parameters (channel coefficients, decay rates, etc.)
  P(θ)     — Prior: our belief about θ BEFORE seeing data
  P(Data|θ)— Likelihood: probability of the observed data given θ
  P(θ|Data)— Posterior: updated belief about θ AFTER seeing data
```

> Key insight: The posterior is a compromise between the prior and the likelihood. With a lot of data the likelihood dominates; with little data the prior plays a larger role.

2.2 Prior Distributions

A prior distribution encodes our beliefs about a parameter before observing data. In the MMM context:

Parameter	Prior & Rationale
intercept	pm.Normal(mu=0, sigma=1) — centred at zero; allows positive or negative baseline
beta_channel	pm.HalfNormal(sigma=1) — constrained positive; channels cannot have negative returns
sigma (noise)	pm.HalfNormal(sigma=1) — positive noise standard deviation
adstock alpha	pm.Beta(alpha=1, beta=1) — uniform over [0,1], no prior preference
saturation K	pm.HalfNormal(sigma=1) — positive half-saturation point

These are called weakly informative priors — they rule out physically impossible values (e.g. negative spend coefficients) while placing minimal constraints on plausible values.

2.3 Likelihood Function

The likelihood function specifies how the model generates data. The MMM uses a Normal likelihood:

```
target_t ~ Normal(mu_t, sigma)

mu_t = intercept + Σ_c [ beta_c × saturation(adstock(spend_c,t)) ]
```

```
Where the sum is over all media channels c.
```

2.4 Posterior Inference via MCMC

The posterior P(θ | Data) rarely has a closed-form solution for complex models. Markov Chain Monte Carlo (MCMC) sampling approximates it by drawing samples from the posterior distribution.

No-U-Turn Sampler (NUTS)

PyMC uses the NUTS algorithm — an extension of Hamiltonian Monte Carlo (HMC) that automatically tunes step sizes and trajectory lengths. NUTS is the gold standard for continuous, high-dimensional posteriors.

NUTS Parameter	Description
draws	Number of posterior samples to retain per chain (default: 1000). More draws → smoother estimates.
tune	Warm-up (burn-in) steps used to adapt the sampler (default: 500). Discarded.
chains	Number of independent Markov chains run in parallel (default: 2). Used to assess convergence.
target_accept	Target acceptance rate (default: 0.9). Higher → smaller step size → more accurate but slower.

Convergence Diagnostics

Before trusting posterior samples, we verify convergence using:

- $\hat{R}$ (R-hat): Compares within-chain and between-chain variance. Values close to 1.0 (< 1.05) indicate convergence.
- Effective Sample Size (ESS): The number of independent samples equivalent to the correlated MCMC draws. Should be > 400.
- Trace Plots: Visual inspection of chains — well-mixed chains that look like 'fuzzy caterpillars' indicate good mixing.

Warning: If $\hat{R}$ > 1.1 for any parameter, the model has not converged. Increase tune/draws or reparameterise the model.

2.5 Posterior Predictive Checks

A posterior predictive check (PPC) tests whether the fitted model can reproduce the observed data. We simulate new datasets from the posterior and compare them to actual observations. If simulated data systematically differ from observed, the model is misspecified.

The engine provides PPC plots in the Insights tab via arviz.plot_ppc().

2.6 Model Comparison Metrics

Metric	Description
WAIC (Widely Applicable Information Criterion)	Estimates out-of-sample predictive accuracy. Lower WAIC = better model. Accounts for effective number of parameters.
LOO-CV (Leave-One-Out Cross Validation)	Approximates leave-one-out cross-validation using Pareto Smoothed Importance Sampling (PSIS-LOO). More robust than WAIC for misspecified models.
ELPD (Expected Log Predictive Density)	The underlying quantity being estimated by WAIC and LOO. Higher = better predictive accuracy.

3. PyMC and PyMC-Marketing

3.1 PyMC — Probabilistic Programming

PyMC is an open-source Python library for Bayesian statistical modelling. It provides:

- A declarative model specification language using Python syntax and NumPy/Aesara arrays.
- State-of-the-art MCMC samplers (NUTS, Metropolis, SMC).
- Automatic differentiation for gradient-based samplers.
- Integration with ArviZ for posterior analysis and diagnostics.

Example: Defining a PyMC model

```
import pymc as pm

with pm.Model() as model:
    # Priors
    intercept = pm.Normal('intercept', mu=0, sigma=1)
    beta      = pm.HalfNormal('beta', sigma=1)
    sigma     = pm.HalfNormal('sigma', sigma=1)

    # Likelihood
    mu = intercept + beta * X
    y_obs = pm.Normal('y_obs', mu=mu, sigma=sigma, observed=y)

    # Sample posterior
    idata = pm.sample(draws=1000, tune=500, chains=2, target_accept=0.9)
```

3.2 ArviZ — Posterior Analysis

ArviZ is the companion library to PyMC for posterior analysis. It operates on InferenceData objects — a standardised container for MCMC outputs. Key capabilities used in the engine:

Function	Purpose
az.plot_trace(idata)	Multi-panel trace and density plots for each parameter.
az.plot_ppc(idata)	Posterior predictive check: simulated vs. observed distributions.
az.waic(idata)	Compute WAIC model comparison metric.
az.loo(idata)	Compute PSIS-LOO model comparison metric.
az.summary(idata)	Table of parameter means, std, HDI, $\hat{R}$, and ESS.

3.3 PyMC-Marketing — MMM Layer

PyMC-Marketing is Pymc's official marketing analytics extension. It provides high-level abstractions that encode marketing domain knowledge into Bayesian models.

The MMM Class

```
from pymc_marketing.mmm import MMM

mmm = MMM(
    date_column     = 'date',
    target_column   = 'clicks',
    channel_columns = ['tv', 'digital', 'radio'],
    adstock_max_lag = 4,
    model_config    = { ... }
)

idata = mmm.fit(df, draws=1000, tune=500, chains=2)
```

Key MMM Methods

Method	Description
mmm.fit(df, **kwargs)	Runs MCMC. Returns ArviZ InferenceData (idata). Internally builds the PyMC model graph, applies adstock transformations, saturation transformations, and samples the posterior.
mmm.predict(df)	Draws samples from the posterior predictive distribution for new/existing data. Returns array of shape (chains × draws × time_periods).
mmm.compute_channel_contribution()	Computes the posterior contribution of each channel to the target variable. Returns xarray DataArray with dims [chain, draw, date, channel].
mmm.plot_response_curves()	Visualises the fitted saturation/Hill transformation curves for each channel, showing diminishing returns.

BudgetOptimizer

The BudgetOptimizer class wraps scipy.optimize to solve a constrained optimisation problem using the fitted MMM's response curves:

```
Maximise:    Σ_c f_c(x_c)          # sum of channel response functions

Subject to:  Σ_c x_c = total_budget # budget constraint
             lb_c ≤ x_c ≤ ub_c      # per-channel bounds

Where f_c(·) is the posterior mean response function for channel c.
```

The optimiser uses Sequential Least Squares Programming (SLSQP) — a gradient-based method well-suited to smooth, differentiable objectives with linear equality constraints.

PART 2 — Application Architecture

4. System Design & Architecture

4.1 Design Principles

- Separation of Concerns: Each Python module has exactly one responsibility. Data loading, validation, modelling, inference, risk, and UI are fully decoupled.
- Configuration Over Code: All tunable parameters (column names, MCMC settings, budget bounds) are controlled via environment variables and a typed Settings class — zero hard-coded values.
- Fail Fast with Clarity: Input validation raises descriptive DataValidationError exceptions that the UI surfaces directly to users — no cryptic stack traces.
- Observability: Every major operation is timed and logged with structured timestamps via a centralised get_logger(__name__) utility.
- Testability: No module imports Streamlit. Business logic is tested without a browser — pure Python unit tests.

4.2 Project Structure

```
mmm_prod/
├── app.py                      ← Streamlit entry point (5-tab UI)
├── config/
│   ├── __init__.py
│   └── settings.py             ← Env-var driven, typed, cached singleton
├── src/
│   ├── data/
│   │   ├── loader.py           ← CSV / Excel file loading
│   │   ├── preprocessor.py     ← Long→wide pivoting & aggregation
│   │   └── validator.py        ← Input validation & error types
│   ├── model/
│   │   ├── trainer.py          ← MMM fitting (MCMC wrapper)
│   │   └── evaluator.py        ← R², MAPE, RMSE, WAIC, LOO-CV
│   ├── inference/
│   │   ├── contributions.py    ← Channel contribution plots
│   │   ├── diagnostics.py      ← Trace plots, PPC, response curves
│   │   └── roas.py             ← ROAS & marginal ROAS computation
│   ├── optimization/
│   │   ├── constraints.py      ← Per-channel budget bounds builder
│   │   └── optimizer.py        ← BudgetOptimizer wrapper
│   ├── scenario/
│   │   └── planner.py          ← Multi-period what-if simulations
│   ├── risk/
│   │   └── uncertainty.py      ← VaR, CVaR, uncertainty summaries
│   └── export/
│       └── exporter.py         ← JSON, CSV, pickle export
├── ui/
│   └── components/             ← One Streamlit component per feature tab
├── utils/
│   ├── logger.py               ← Centralised logging
│   └── helpers.py              ← timer(), safe_divide(), flatten_dict()
├── tests/                      ← 60 pytest tests across 5 modules
├── .env.example
```

```
├── Makefile
├── pytest.ini
└── requirements.txt
```

4.3 Data Flow

The application follows a strict unidirectional data flow through six stages:

```
[1] Upload          → load_file()       → raw pd.DataFrame
[2] Validate        → validate_raw()    → DataValidationError or pass
[3] Preprocess      → prepare_data()    → wide model_df + channel_cols
[4] Train           → train_mmm()       → MMM object + InferenceData
[5] Analyse         → inference/*       → plots + tables + metrics
[6] Decide          → optimizer/* or    → allocation DataFrame
                       scenario/*       → forecast DataFrame
```

> All intermediate results are stored in st.session_state — a Streamlit-managed dictionary. This persists values across tab navigation without global variables or file I/O.

5. Module-by-Module Reference

5.1 config/settings.py — Configuration Management

The Settings dataclass is the single source of truth for all runtime parameters. It is instantiated once at startup via @lru_cache and reused everywhere.

```
from config import get_settings
cfg = get_settings()

cfg.data.target_col        # e.g. 'clicks'
cfg.model.draws            # e.g. 1000
cfg.optimization.default_budget  # e.g. 100000.0
```

Setting	Env Variable	Default
DataConfig.date_col	DATE_COL	Cal_Month
DataConfig.target_col	TARGET_COL	clicks
DataConfig.spend_col	SPEND_COL	media_spend
DataConfig.channel_col	CHANNEL_COL	site
ModelConfig.draws	MCMC_DRAWS	1000
ModelConfig.tune	MCMC_TUNE	500
ModelConfig.chains	MCMC_CHAINS	2
ModelConfig.adstock_max_lag	ADSTOCK_MAX_LAG	4
OptimizationConfig.default_budget	DEFAULT_BUDGET	100000
OptimizationConfig.min_channel_share	MIN_CHANNEL_SHARE	0.05
OptimizationConfig.max_channel_share	MAX_CHANNEL_SHARE	0.60
ExportConfig.output_dir	OUTPUT_DIR	outputs

5.2 src/data/ — Data Pipeline

loader.py — load_file(source)

Accepts a file path (str/Path) or a file-like object (Streamlit UploadedFile, BytesIO). Dispatches to pd.read_csv() or pd.read_excel() based on file extension. Raises IOError for missing files and ValueError for unsupported extensions.

validator.py — validate_raw(df) / validate_model_df(df, channel_cols)

validate_raw() enforces:

- All four required columns present (date, channel, spend, target).
- Spend and target columns are numeric dtype.

- No negative spend or target values.
- No null values in required columns.

validate_model_df() enforces:

- 'date' and target columns are present after pivoting.
- All channel columns exist.
- No negative spend values in channel columns.

preprocessor.py — prepare_data(df) → (model_df, channel_cols)

Transforms the raw long-format DataFrame into the wide format required by PyMC-Marketing's MMM. The pipeline:

1. Parse the date column with pd.to_datetime().
2. Group by (date, channel) and sum both spend and target.
3. Pivot the spend column: rows = dates, columns = channels, values = spend.
4. Re-attach the aggregated target series.
5. Sort by date and reset the index.
6. Extract channel_cols: all columns except 'date' and target.

5.3 src/model/ — Model Training & Evaluation

trainer.py — train_mmm(df, channel_cols) → (mmm, idata)

Constructs and fits the Bayesian MMM. The function:

7. Assembles the model_config dict with weakly informative priors.
8. Instantiates MMM with date_column, target_column, channel_columns, adstock_max_lag.
9. Calls mmm.fit(df) with MCMC settings pulled from config (draws, tune, chains, target_accept, random_seed).
10. Returns the fitted MMM object and the ArviZ InferenceData.

evaluator.py — full_evaluation_report(mmm, idata, df, target_col)

Aggregates three families of metrics into a single report dict:

Metric Key	Interpretation
in_sample.r2	Coefficient of determination. 1.0 = perfect fit. Uses posterior mean predictions vs. observed.
in_sample.mape	Mean Absolute Percentage Error. Lower = better. Sensitive to near-zero values.
in_sample.rmse	Root Mean Squared Error in the units of the target variable.
waic.waic	WAIC estimate. Lower = better out-of-sample predictive accuracy.
waic.waic_se	Standard error of the WAIC estimate.

Metric Key	Interpretation
loo.elpd_loo	Expected log predictive density from LOO-CV. Higher = better.
loo.se	Standard error of the LOO ELPD estimate.

5.4 src/inference/ — Posterior Analysis

contributions.py

Provides three functions:

- get_channel_contributions(mmm): Calls mmm.compute_channel_contribution() → xr.DataArray of shape [chain, draw, date, channel].
- contributions_to_dataframe(contrib): Collapses posterior via .mean(dim=['chain','draw']) → pd.DataFrame with dates as index and channels as columns.
- plot_channel_contributions(contrib): Bar chart of total posterior-mean contribution per channel.
- plot_contribution_over_time(contrib): Stacked area chart showing channel contributions across time periods.

diagnostics.py

- plot_posterior_diagnostics(idata): ArviZ trace plots. Each parameter gets a density plot and a time-series plot of samples across draws. Well-mixed chains look like 'fuzzy caterpillars'.
- plot_response_curves(mmm): PyMC-Marketing's built-in response curve visualisation showing the Hill/saturation function fitted per channel.
- plot_posterior_predictive_check(mmm, idata, target_col): Overlay of posterior predictive samples over the observed target distribution.

roas.py

- compute_marginal_roas(mmm): Returns a pd.Series of posterior-mean contributions per channel (a proxy for marginal ROAS).
- compute_roas(mmm, df, channel_cols): Full ROAS table with columns [channel, total_spend, total_contribution, roas]. ROAS = total_contribution / total_spend.

5.5 src/optimization/ — Budget Optimisation

constraints.py — build_constraints(channel_cols, overrides)

Constructs a dict of {channel: (min_share, max_share)} bounds. Validates:

- Each bound is within [0, 1].
- min_share ≤ max_share for each channel.
- Sum of all min_share values ≤ 1.0 (feasibility check).

optimizer.py — run_optimization(mmm, total_budget, bounds) → pd.DataFrame

Wraps BudgetOptimizer.optimize(). Handles three types of raw output (dict, pd.Series, pd.DataFrame) from different versions of PyMC-Marketing and normalises them into a tidy result with columns [channel, optimized_spend, share].

5.6 src/scenario/ — Scenario Planning

planner.py — generate_scenario(df, multipliers, periods) → pd.DataFrame

Extends the historical DataFrame by appending periods future rows. Each row is built by:

11. Taking the last historical row as a template.
12. Adding exactly one month to the date (pd.DateOffset(months=1)).
13. Applying each channel multiplier: new_spend = old_spend × multiplier.
14. Clamping negative results to 0 (spend cannot go negative).

simulate_scenario(mmm, scenario_df) → array

Calls mmm.predict(scenario_df) to sample from the posterior predictive distribution for the scenario rows. Returns a 3D array of shape (chains × draws × time_periods).

build_scenario_summary(scenario_df, preds, target_col, channel_cols) → pd.DataFrame

Aligns predictions with the scenario DataFrame. Adds columns predicted_mean, predicted_p5, predicted_p95 computed from the posterior predictive array.

5.7 src/risk/ — Uncertainty Quantification

uncertainty.py

All functions accept raw prediction arrays of any shape (flattened with np.ravel internally).

Function	Description
summarize_with_uncertainty(predictions, percentiles)	Returns dict with mean, std, and p{x} for each requested percentile. Default percentiles: (5, 25, 50, 75, 95).
value_at_risk(predictions, confidence=0.05)	Returns the lower-tail threshold: 5% of outcomes are below this value. Answers: 'In the worst 5% of outcomes, what is the minimum result?'
conditional_value_at_risk(predictions, confidence=0.05)	Expected Shortfall: the average outcome in the worst 5% of cases. Always ≤ VaR. More conservative risk measure.
plot_prediction_distribution(predictions)	Histogram with vertical lines at P5, P50, P95. Visualises the full posterior predictive distribution.
build_risk_report(predictions)	Aggregates all metrics into a single report dict with keys: summary, var_5pct, cvar_5pct.

5.8 src/export/ — Results Export

exporter.py

Function	Description
export_json(data, filename, output_dir, timestamp=True)	Serialises any dict containing DataFrames, Series, or numpy types to JSON. Timestamps filenames to avoid overwrites. Uses _make_serialisable() for type conversion.
export_csv(df, filename, output_dir, timestamp=True)	Writes a DataFrame to CSV. Adds ISO timestamp prefix by default.
export_model(mmm, filename, output_dir)	Pickles the fitted MMM object to disk for later reuse.
load_model(path)	Unpickles and returns a previously saved MMM.
_make_serialisable(obj)	Recursively converts numpy.int64/float32, np.ndarray, pd.DataFrame, pd.Series to JSON-safe Python types.

6. UI Components — Streamlit Interface

The UI is structured as a 5-tab Streamlit application. Each tab maps to an independent component in ui/components/. Components communicate exclusively through st.session_state — they never import from each other.

6.1 Session State Schema

Key	Value & Type
raw_df	pd.DataFrame — the original uploaded file as-is.
model_df	pd.DataFrame — wide-format model-ready data.
channel_cols	List[str] — media channel column names.
mmm	MMM — fitted PyMC-Marketing model object.
idata	InferenceData — ArviZ MCMC output.
eval_report	Dict — model evaluation metrics (R^2, WAIC, etc.).
mcmc_draws	int — MCMC draws set in sidebar.
mcmc_chains	int — MCMC chains set in sidebar.
adstock_lag	int — adstock max lag set in sidebar.

6.2 Component Summary

Component	Functionality
sidebar.py — render_sidebar()	Global controls: MCMC draws/chains/lag sliders, session state indicators, reset button. Persists settings to st.session_state.
data_upload.py — render_data_upload()	File uploader widget, raw data preview with row/col/null metrics, column summary expander, preprocessed wide-format preview.
model_training.py — render_model_training()	Training config display, Train button with progress bar, evaluation metrics panel (R^2, MAPE, RMSE, WAIC, LOO).
insights.py — render_insights()	Channel contribution bar & area charts, ROAS table, response curves, trace plots, posterior predictive checks.
scenario_planner.py — render_scenario_planner()	Per-channel multiplier sliders, period selector, forecast table for future rows, prediction distribution plot, uncertainty summary metrics.
optimizer_ui.py — render_optimizer()	Budget input, per-channel share constraint expanders, optimised allocation table and bar chart, risk metrics, JSON/CSV export buttons.

7. Utilities

7.1 utils/logger.py — Centralised Logging

Every module obtains a logger via get_logger(__name__). This ensures all log lines share a consistent format and are routed to stdout. The format is:

```
2026-04-11 14:23:01 | INFO     | src.model.trainer | MMM training complete.
```

Log levels follow Python's standard hierarchy: DEBUG < INFO < WARNING < ERROR < CRITICAL. The default level is INFO, controllable via the LOG_LEVEL environment variable.

7.2 utils/helpers.py

Function	Description
timer(label)	Context manager. Logs elapsed time in seconds when the block exits. Used around every major operation (training, optimization, scenario simulation).
safe_divide(numerator, denominator, fallback=0.0)	Division with zero-denominator guard. Used in ROAS calculation to avoid ZeroDivisionError when a channel has zero spend.
flatten_dict(d, sep='.')	Recursively flattens nested dicts to a single level using dot-separated keys. Useful for logging nested config or exporting nested results.

8. Test Suite

The test suite contains 60 pytest tests across 5 modules. All tests operate on pure Python logic — no Streamlit context, no MCMC sampling, no external services required.

8.1 Test Coverage Summary

Module	Count	Tests Cover
tests/test_preprocessing.py	16 tests	validate_raw, prepare_data, validate_model_df
tests/test_constraints.py	9 tests	build_constraints, bound validation, feasibility
tests/test_risk.py	11 tests	summarize_with_uncertainty, VaR, CVaR, risk report
tests/test_scenario.py	11 tests	generate_scenario, build_scenario_summary
tests/test_export.py	13 tests	_make_serialisable, export_json, export_csv

8.2 Running Tests

```
# Full suite
make test

# Fast mode (stop on first failure)
make test-fast

# Specific module
pytest tests/test_preprocessing.py -v

# With coverage report
pytest --cov=src --cov-report=html
# Then open htmlcov/index.html
```

8.3 Test Design Patterns

- Fixtures: @pytest.fixture creates reusable DataFrames (valid_raw_df, base_df) with controlled, reproducible data.
- Class grouping: Tests are grouped in classes (TestValidateRaw, TestPrepareData) for clarity and shared setup.
- Edge cases: Every validator function is tested with both valid inputs (should not raise) and invalid inputs (should raise the correct exception type and message).
- Numerical assertions: pytest.approx() is used for floating-point comparisons to avoid brittle exact equality.
- Determinism: np.random.default_rng(42) ensures test arrays are reproducible.

9. Configuration & Deployment

9.1 Environment Variables (.env.example)

Copy .env.example to .env and adjust for your dataset and infrastructure. The application reads all settings at startup via Python's os.getenv() — no .env loading library is required (use python-dotenv or export manually in CI).

```
# Data column mappings
DATE_COL=Cal_Month
TARGET_COL=clicks
SPEND_COL=media_spend
CHANNEL_COL=site

# MCMC settings
MCMC_DRAWS=1000
MCMC_TUNE=500
MCMC_CHAINS=2
MCMC_TARGET_ACCEPT=0.9
RANDOM_SEED=42

# Budget optimisation
DEFAULT_BUDGET=100000
MIN_CHANNEL_SHARE=0.05
MAX_CHANNEL_SHARE=0.60

# App
LOG_LEVEL=INFO
DEBUG=false
OUTPUT_DIR=outputs
```

9.2 Installation

```
# 1. Clone the repository
git clone <repo-url> && cd mmm_prod

# 2. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate    # Linux/Mac
.venv\Scripts\activate       # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure
cp .env.example .env
# Edit .env to match your column names

# 5. Launch
streamlit run app.py
```

9.3 Makefile Commands

Command	Action
make install	pip install -r requirements.txt
make run	streamlit run app.py
make test	Full pytest suite with coverage
make test-fast	Fail-fast pytest, no HTML coverage
make lint	ruff check — static analysis
make format	ruff format — auto-formatting
make clean	Remove __pycache__, .pytest_cache, outputs/

10. Step-by-Step Usage Guide

10.1 Preparing Your Data

The application expects long-format (tidy) tabular data where each row represents one channel's spend in one time period. Required columns:

Column	Requirements
Date column (default: Cal_Month)	One row per time period per channel. Any date format parseable by pandas — e.g. '2024-01', '01/01/2024'.
Channel column (default: site)	String identifier for the media channel — e.g. 'TV', 'Digital', 'Radio'.
Spend column (default: media_spend)	Numeric. Total spend for that channel in that period. Must be non-negative.
Target column (default: clicks)	Numeric. The KPI to model — clicks, revenue, conversions. Must be non-negative.

Tip: If your column names differ, update DATE_COL, TARGET_COL, SPEND_COL, CHANNEL_COL in your .env file before launching.

10.2 Training the Model

15. Upload your CSV or Excel file in the Data tab.
16. Verify the data preview — check row counts, null values, and the wide-format output.
17. Adjust MCMC settings in the sidebar: increase Draws to 2000+ for production; use 200-500 for quick testing.
18. Navigate to the Train tab and click 'Train MMM'.
19. Monitor the progress bar and review evaluation metrics. $R^2 > 0.85$ and $\hat{R} \approx 1.0$ are good signs.

Performance note: Training with draws=1000, chains=2 on a typical marketing dataset (2 years monthly, 5 channels) takes approximately 3-8 minutes on a modern CPU. Enable GPU support via JAX for faster sampling.

10.3 Interpreting Insights

20. Channel Contributions Bar Chart: Higher bars = more attributable sales/clicks. Channels with low contribution relative to spend are candidates for reallocation.
21. ROAS Table: Sort by ROAS column. High ROAS channels should receive higher budget share in the optimiser. Channels with ROAS < 1 are potentially unprofitable.
22. Response Curves: Look for the inflection point where the curve flattens. Spend beyond this point yields sharply diminishing returns.
23. Trace Plots: All chains should overlap and show no trends. If a chain diverges, the model has not converged — increase tune.

24. Posterior Predictive Check: The simulated distribution (light bands) should envelope the observed data. Systematic gaps indicate model misfit.

10.4 Running Scenarios

25. Navigate to the Scenarios tab.
26. Set multipliers for each channel: 0.5 = halve spend, 1.0 = baseline, 2.0 = double spend.
27. Set the number of forecast periods (months).
28. Click 'Run Scenario'. The model generates posterior predictive samples for the modified spend plan.
29. Review the forecast table (predicted_mean, p5, p95 per period) and the distribution plot.
30. Use the P5/P95 credible interval as a downside/upside risk range for planning.

10.5 Optimising the Budget

31. Navigate to the Optimise tab.
32. Enter the total budget to allocate.
33. Optionally expand each channel's constraint panel to set custom min/max share bounds.
34. Click 'Optimise Budget'. The solver maximises the expected total outcome subject to the budget constraint.
35. Review the allocation table (optimized_spend, share columns) and bar chart.
36. Check the risk panel: VaR and CVaR give downside exposure under the optimal allocation.
37. Export results to JSON or CSV for reporting.

11. Glossary

Term	Definition
Adstock	Weighted cumulative effect of past advertising spend. Models carryover/memory effect of media exposure.
ArviZ	Python library for Bayesian model diagnostics. Provides trace plots, PPC, WAIC, and LOO.
Baseline	Sales or conversions that occur without any media investment. Driven by organic demand and seasonality.
Bayesian Inference	Statistical framework that updates prior beliefs with data to produce posterior distributions.
Beta Distribution	Continuous distribution over [0,1]. Used as prior for decay rates (alpha).
Carryover Effect	The persistence of advertising impact beyond the immediate period of exposure. See Adstock.
Chains	Independent MCMC sampling runs. Multiple chains allow convergence assessment via R-hat.
Credible Interval	Bayesian equivalent of confidence interval: the range containing X% of the posterior probability mass. Has a direct probability interpretation.
CVaR (Conditional VaR)	Expected Shortfall. The mean outcome in the worst-case tail. Always ≤ VaR.
DataValidationError	Custom exception raised by the validation module when input data fails checks.
Diminishing Returns	The saturation effect: each additional unit of spend yields progressively less incremental outcome.
Draws	Retained posterior samples per MCMC chain. More draws = smoother posterior approximation.
ESS (Effective Sample Size)	The number of independent samples equivalent to correlated MCMC draws. Should exceed 400 per parameter.
HalfNormal Distribution	Normal distribution truncated at zero. Used as prior for positive-only parameters (coefficients, noise).
Hill Transformation	S-shaped saturation function parameterised by shape (alpha) and half-saturation (K). See Saturation.
InferenceData	ArviZ container for MCMC outputs: posterior, posterior_predictive, observed_data, sample_stats.

Term	Definition
LOO-CV	Leave-One-Out Cross Validation. Approximated with PSIS for computational efficiency. Estimates out-of-sample predictive accuracy.
Marginal ROAS	The additional outcome generated by spending one more monetary unit in a channel. Derived from the slope of the response curve.
MCMC	Markov Chain Monte Carlo. Family of algorithms for sampling from complex probability distributions.
MMM	Marketing Mix Modelling. Statistical method for attributing business outcomes to marketing channels.
NUTS	No-U-Turn Sampler. PyMC's default MCMC algorithm. Automatically tunes trajectory length.
PPC	Posterior Predictive Check. Validates model fit by comparing simulated data to observed data.
Prior Distribution	Probability distribution encoding beliefs about a parameter before observing data.
Posterior Distribution	Updated belief about a parameter after combining prior with likelihood via Bayes' Theorem.
R-hat ($\hat{R}$)	Gelman-Rubin convergence diagnostic. Values < 1.05 indicate chain convergence.
ROAS	Return on Ad Spend. Total outcome attributable to a channel divided by its total spend.
Response Curve	Plot of marginal or total outcome vs. spend level for a channel. Shows saturation point.
Saturation	Non-linear transformation modelling diminishing returns at high spend levels. See Hill Transformation.
Session State	Streamlit's st.session_state dictionary. Persists data across tab navigation within a session.
SLSQP	Sequential Least Squares Programming. Gradient-based optimisation algorithm used by BudgetOptimizer.
Tune	MCMC warm-up steps used to adapt the sampler. Discarded from the posterior.
VaR (Value at Risk)	The threshold below which X% of outcomes fall. A lower-tail risk measure.
WAIC	Widely Applicable Information Criterion. Penalised log-likelihood for model comparison. Lower = better.
xarray DataArray	N-dimensional labelled array. Used by PyMC-Marketing to store posterior samples with dim/coord labels.

12. Quick Reference Card

Public API — One-Line Summaries

```
# Data
load_file(source)                                  → pd.DataFrame
prepare_data(df)                                   → (model_df, channel_cols)
validate_raw(df)                                   → None | raises DataValidationError

# Model
train_mmm(df, channel_cols)                        → (MMM, InferenceData)
full_evaluation_report(mmm, idata, df, target_col) → dict

# Inference
get_channel_contributions(mmm)                     → xr.DataArray
contributions_to_dataframe(contrib)                → pd.DataFrame
compute_roas(mmm, df, channel_cols)                → pd.DataFrame
plot_channel_contributions(contrib)                → plt.Figure
plot_contribution_over_time(contrib)               → plt.Figure
plot_response_curves(mmm)                          → plt.Figure
plot_posterior_diagnostics(idata)                  → plt.Figure
plot_posterior_predictive_check(mmm, idata, col)   → plt.Figure

# Optimisation
build_constraints(channel_cols, overrides)         → BoundsDict
run_optimization(mmm, total_budget, bounds)        → pd.DataFrame

# Scenario
generate_scenario(df, multipliers, periods)        → pd.DataFrame
simulate_scenario(mmm, scenario_df)                → np.ndarray
build_scenario_summary(scenario_df, preds, ...)    → pd.DataFrame

# Risk
summarize_with_uncertainty(predictions)            → dict
value_at_risk(predictions, confidence=0.05)        → float
conditional_value_at_risk(predictions, 0.05)       → float
build_risk_report(predictions)                     → dict

# Export
export_json(data, filename, output_dir)            → Path
export_csv(df, filename, output_dir)               → Path
export_model(mmm, filename, output_dir)            → Path
load_model(path)                                   → MMM
```

Common Troubleshooting

Symptom	Resolution
$\hat{R}$ > 1.1 for some parameters	Increase tune to 1000+ and draws to 2000+. Consider reparameterising priors. Check for multicollinearity between channels.
'Missing required columns' error	Update DATE_COL / TARGET_COL / SPEND_COL / CHANNEL_COL in .env to match actual column names.

Symptom	Resolution
Optimiser returns equal allocation	The response curves may be very similar across channels. Increase MCMC draws for more precise posteriors or add tighter channel constraints.
Training takes > 30 minutes	Reduce draws to 500 for exploration. Ensure multiprocessing is available (chains > 1). Use JAX backend for GPU support.
Scenario predictions are very wide	Normal with small datasets — the posterior has high uncertainty. Collect more data or add stronger informative priors.
CVaR > VaR assertion fails	Theoretical impossibility — indicates a bug in the prediction array shape. Ensure predictions are (chains × draws × time).